

Innehåll

1. [Installera Yarn extension i ett Azure DevOps](#)
2. [Bygga en build pipeline med hjälp av YAML](#)

Yarn

Yarn är ett pakethanteringssystem som används för att automatisera processen att installera, uppdatera, konfigurera och ta bort kodpaket från ett projekt. Det skapades som ett alternativ till Node.js-pakethanteraren npm (Node Package Manager) och har några fördelar i form av hastighet, pålitlighet och säkerhet. Här är några nyckelpunkter om Yarn:

1. **Hantering av Beroenden:** Yarn gör det enkelt att lägga till, uppgradera, konfigurera och ta bort tredjepartspaket som ditt projekt är beroende av.
2. **Effektivitet:** Det är snabbare än npm vid installation av paket tack vare en mekanism som cachar varje nedladdat paket och parallelliserar operationer.
3. **Säkerhet:** Yarn använder en detaljerad `yarn.lock`-fil som låser versionerna av varje beroende för att undvika problem med olika versioner.
4. **Kompatibilitet:** Yarn är kompatibelt med npm-registret, vilket innebär att du kan använda paket från npm:s omfattande ekosystem.
5. **Workspaces:** Stöder arbetsytor som är användbara i monorepos, vilket gör det enklare att hantera projekt med flera paket.

Kort sagt är Yarn ett kraftfullt verktyg för JavaScript-utvecklare som underlättar hanteringen av projektberoenden och förbättrar arbetsflödena vid programutveckling.

Att installera Yarn extension i ett Azure DevOps organisation kräver några enkla steg. Här är en steg-för-steg guide:

Steg 1: Logga in på Azure DevOps

- Besök [Azure DevOps](#) och logga in med dina Azure DevOps-organisationsuppgifter.

Steg 2: Navigera till Azure DevOps Marketplace

- Öppna en ny flik i din webbläsare och besök [Azure DevOps Marketplace](#).

Steg 3: Sök efter Yarn Extension

- Använd sökfältet i Azure Marketplace och skriv in "Yarn".
- Bläddra igenom sökresultaten tills du hittar Yarn extension.

Steg 4: Välj Yarn Extension

- Klicka på Yarn extension för att öppna dess detaljsida.

Steg 5: Installera Extension

- På detaljsidan, klicka på knappen "Get it free" eller "Install".
- Du kommer att bli ombedd att välja din Azure DevOps-organisation där du vill installera extensionen.

Steg 6: Välj Organisation

- Välj organisationen där du vill installera Yarn.
- Klicka sedan på "Install".

Steg 7: Bekräfta Installationen

- Du kanske behöver godkänna vissa behörigheter eller bekräfta installationen.
- Följ eventuella ytterligare instruktioner som dyker upp för att slutföra processen.

Steg 8: Verifiera Installationen

- Gå tillbaka till din Azure DevOps-organisation.
- Gå till "Project settings" och sedan till "Extensions" för att se att Yarn extension nu är listad där.

Steg 9: Använd Yarn i Dina Pipelines

- Nu när Yarn är installerad, kan du använda den i dina build pipelines genom att referera till den i dina `.yaml`-filer.

Steg 10: Konfigurera Yarn i Dina Build Scripts

- I dina build scripts (.yaml-filer), lägg till steg för att använda Yarn för att installera beroenden och köra byggprocesser.

Glöm inte att dessa steg kan variera något beroende på specifika uppdateringar eller ändringar i Azure DevOps eller Marketplace. Håll alltid din dokumentation och kunskap uppdaterad.

YAML

Att bygga en Build Pipeline i Azure DevOps för ett React-projekt som använder Vite och TypeScript med hjälp av YAML (.yaml) innebär flera steg. Låt oss bryta ner processen:

Steg 1: Skapa en YAML-fil

Först behöver du skapa en `.yaml`-fil i ditt projektrepository. Denna fil kommer att innehålla alla konfigurationer och definitioner för din pipeline.

Steg 2: Definiera Triggers

I din YAML-fil börjar du med att definiera vilka händelser som ska starta byggprocessen. Vanligtvis är detta en push till en specifik branch, som `main`, `master` eller `develop`.

```
trigger:  
- master
```

Steg 3: Ange Agent Pool

Specificera vilken agent som ska användas. Azure DevOps erbjuder flera förkonfigurerade agenter.

```
pool:  
  vmImage: 'ubuntu-latest'
```

Steg 4: Steg för Installation

Definiera stegen för att installera beroenden. För ett Vite, React, TypeScript-projekt innebär detta vanligtvis att installera och köra `npm install` eller `yarn`.

```
- task: NodeTool@0  
  inputs:  
    versionSpec: '20.x'  
    displayName: 'Install Node.js'  
  
- task: YarnInstaller@3  
  displayName: 'Install yarn 1.x'  
  inputs:  
    versionSpec: '1.x'  
  
- script: |  
  yarn install  
  yarn run build  
  displayName: 'yarn install and build'
```

Steg 5: Kör Build Script

Lägg till ett steg för att köra ditt byggsript. Med Vite kan detta vara så enkelt som `npm run build` eller `vite build`.

```
- task: Yarn@3  
  displayName: 'Build'  
  inputs:  
    arguments: 'run build'
```

Steg 7: Artefakter

Definiera hur och var dina byggda artefakter ska lagras. Detta kan vara en dist-mapp som skapas av Vite.

```
- task: ArchiveFiles@2
  inputs:
    rootFolderOrFile: './dist'
    includeRootFolder: false
    archiveType: 'zip'
    archiveFile: '$(Build.ArtifactStagingDirectory)/$(Build.BuildId).zip'
    replaceExistingArchive: true

- task: PublishBuildArtifacts@1
  displayName: 'Publish Artifact: share'
  inputs:
    PathToPublish: '$(Build.ArtifactStagingDirectory)/$(Build.BuildId).zip'
    ArtifactName: chatgpt
```

Steg 8: Ytterligare Konfigurationer

Beroende på projektets krav kan du lägga till ytterligare konfigurationer som miljövariabler, cache-konfigurationer, eller specifika inställningar för TypeScript-kompilering.

Steg 9: Commit och Push

När du är nöjd med din `.yaml`-fil, commita och pusha den till ditt repository. Azure DevOps kommer automatiskt att identifiera filen och börja använda den för byggprocessen.

Steg 10: Monitorera och Felsök

Efter att du har pushat din `.yaml`-fil, gå till Azure DevOps och övervaka din första byggprocess. Se till att felsöka och justera efter behov.

Genom att följa dessa steg kan du effektivt konfigurera en Build Pipeline i Azure DevOps för ditt React-projekt som använder Vite och TypeScript. Varje projekt kan ha unika krav, så anpassa konfigurationen efter dina behov.